## APPENDIX

### A. Additional Time-Space Tradeoff Results

We provide additional time-space cost distributions for ALEX on Covid and LIPP on Book to complement the results in Section IV-C. Specifically, Fig. 13 and Fig. 14 illustrate the average query latency and memory consumption of different configurations under BALANCE, INSERT, INSERT_READ, and READ workloads.

As shown in Fig. 13, Sidecar+ achieves a favorable time-space tradeoff under BALANCE, INSERT, and INSERT_READ. It substantially reduces latency while maintaining memory consumption close to or below the original ALEX. No-Compress and Static-Hash incur higher memory consumption, whereas All-Hash sacrifices memory efficiency and ordered storage for lower latency. Compressed reduces space consumption but incurs substantially higher latency due to frequent key movements.

Similarly, Fig. 14 shows that Sidecar+ achieves a favorable time-space tradeoff under the three update-intensive workloads, reducing both average latency and memory consumption compared with the original LIPP. Compared with No-Compress and Static-Hash, it maintains comparable latency with a smaller memory footprint. Concurrency reduces rebuilding overhead but remains less effective than hash conversion in mitigating persistent structural degradation. Under the read-only workload, the differences among configurations are smaller, as no further insertions trigger structural degradation.

### B. Additional Runtime Adaptation Behavior

We further present the runtime adaptation behavior of ALEX on Covid under the INSERT_READ workload in Figure 15, illustrating the relationship between structural adjustments, latency changes, and memory consumption.

For ALEX on Covid, hash conversions are concentrated in the insertion phase, with latency decreasing stepwise as degraded leaf nodes are converted. After the workload switches to the read phase at interval 200, latency drops sharply and remains stable. The average latency of Sidecar+ is 551 ns, compared with 1,275 ns for the original ALEX, representing a 56.8% reduction. Meanwhile, cold-node compaction continues to reclaim redundant space, whereas No-Compress retains its accumulated memory footprint. These results illustrate the complementary roles of selective hash conversion and cold-node compaction in improving performance and controlling space consumption.

### C. Additional Results under Distribution Shifts

We provide additional results for the two remaining index-dataset combinations, ALEX on Covid and LIPP on Book, to complement the results in Section IV-B. These combinations cover concentrated and dispersed distribution shifts, respectively, allowing us to further examine the effectiveness of Sidecar under different patterns of structural degradation.

For ALEX on Covid, Sidecar+ improves throughput from 14.34 to 22.17 M ops/s (1.55×) under BALANCE, from 9.72 to 15.94 M ops/s (1.64×) under INSERT, and from 13.30 to 21.67 M ops/s (1.63×) under INSERT_READ.

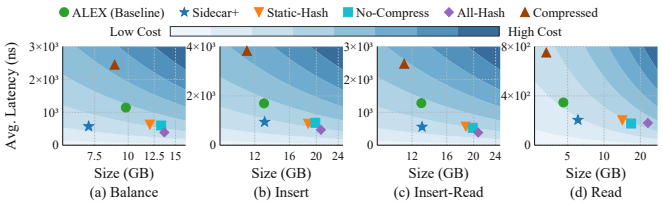



Fig. 13. Time-space cost distributions of ALEX variants on Covid (32 threads).

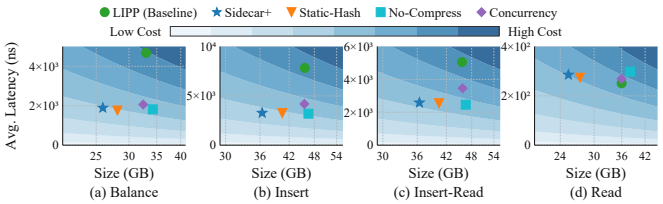



Fig. 14. Time-space cost distributions of LIPP variants on Book (32 threads).

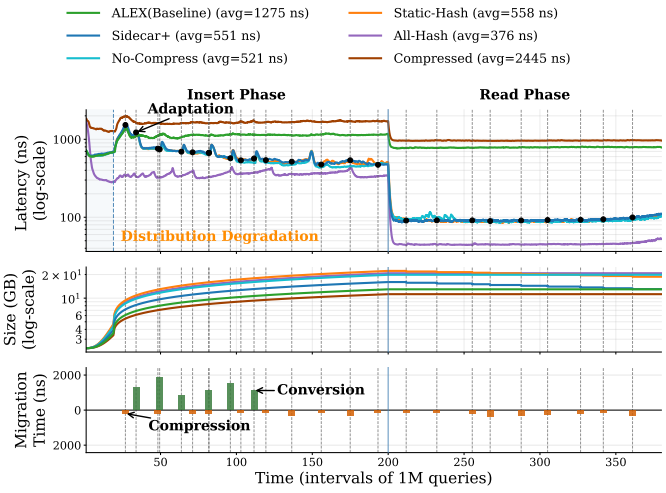
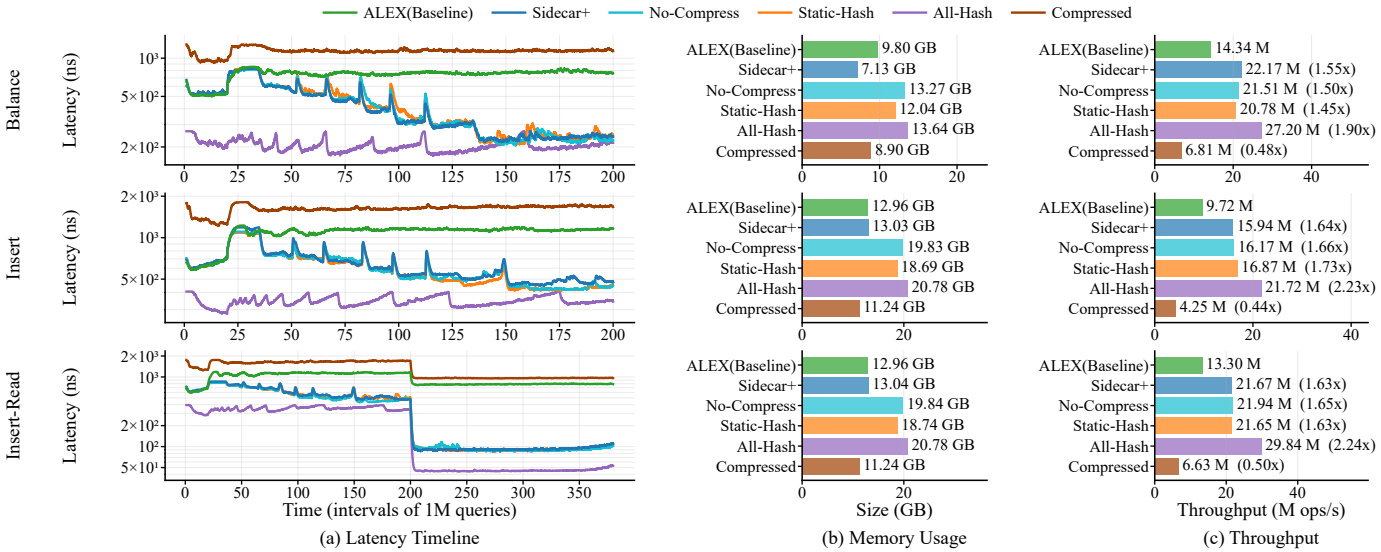



Fig. 15. Runtime adaptation of ALEX on Covid under INSERT_READ, showing latency, memory consumption, and structural adjustment events (32 threads).

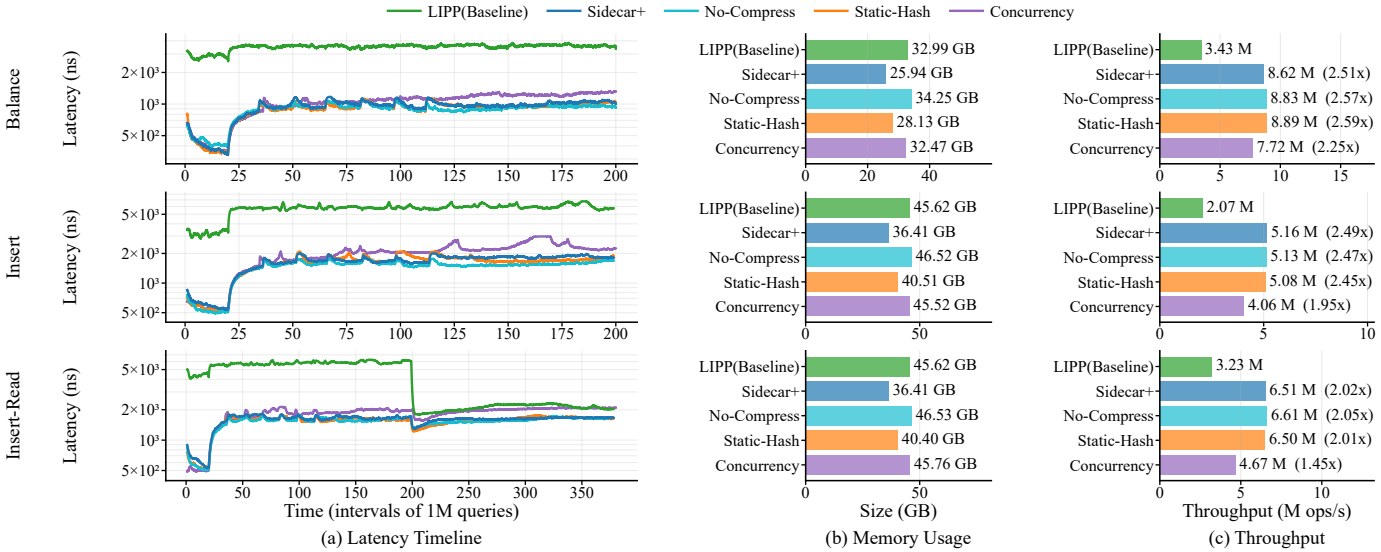
The latency timelines show consistent reductions as degraded leaf nodes are converted. All-Hash achieves higher throughput but sacrifices the ordered organization required for efficient range scans, whereas Compressed exhibits the lowest throughput due to frequent key movements. In terms of memory consumption, Sidecar+ reduces usage from 9.80 to 7.13 GB under BALANCE and maintains memory consumption close to the baseline under INSERT and INSERT_READ. Nevertheless, it uses substantially less memory than No-Compress and Static-Hash.

For LIPP on Book, Sidecar+ increases throughput from 3.43 to 8.62 M ops/s (2.51×) under BALANCE, from 2.07 to 5.16 M ops/s (2.49×) under INSERT, and from 3.23 to 6.51 M ops/s (2.02×) under INSERT_READ. It also reduces latency across all three workloads. Concurrency improves throughput over the original LIPP by decoupling rebuilding decisions, but remains less effective than Sidecar+ under INSERT and




(a) Latency Timeline (b) Memory Usage (c) Throughput

Fig. 16. Latency timelines, memory consumption, and throughput of ALEX variants on Covid (32 threads).




(a) Latency Timeline (b) Memory Usage (c) Throughput

Fig. 17. Latency timelines, memory consumption, and throughput of LIPP variants on Book (32 threads).

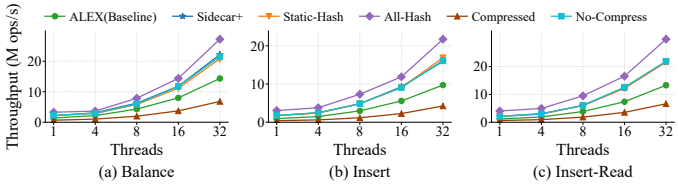



Fig. 18. Throughput scalability of ALEX variants on Covid across 1, 4, 8, 16, and 32 threads.

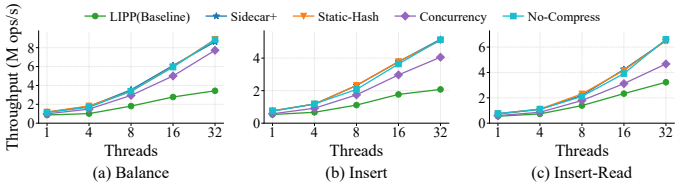



Fig. 19. Throughput scalability of LIPP variants on Book across 1, 4, 8, 16, and 32 threads.

INSERT_READ. Meanwhile, Sidecar+ reduces memory consumption from 32.99 to 25.94 GB under BALANCE and from 45.62 to 36.41 GB under both INSERT and INSERT_READ. The dispersed distribution shifts in Book affect multiple subtrees, making selective subtree hash conversion and cold-node compaction effective in improving performance and reclaiming redundant space.

Together with the results in Section IV-B, these experiments confirm that Sidecar improves throughput and reduces latency across both indexes and distribution-shift patterns. The memory benefits depend on the extent of structural degradation and the available reclamation opportunities. Dispersed shifts in Book cause more widespread degradation, whereas concentrated shifts in Covid affect fewer local regions. Nevertheless, Sidecar effectively controls memory overhead in both cases through dynamic capacity reservation and cold-node compaction.

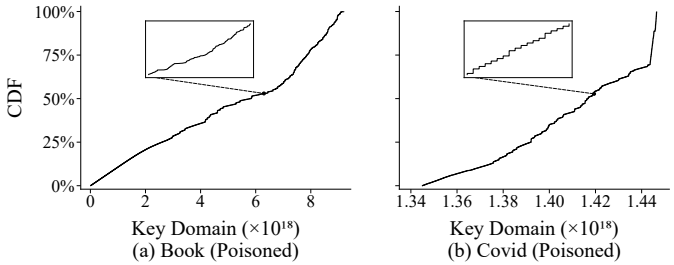



Fig. 20. CDF distributions of Book and Covid after runtime attack-key injection, illustrating dispersed and clustered distribution shifts, respectively.

### *D. Additional Multithreaded Scalability Results*

We provide additional multithreaded scalability results for ALEX on Covid and LIPP on Book to complement the representative results in Section IV-E. The experiments cover BALANCE, INSERT, and INSERT_READ workloads with 1, 4, 8, 16, and 32 threads.

*1) ALEX on Covid:* Figure 18 shows that throughput increases with the number of threads across all three workloads. Sidecar+, Static-Hash, and No-Compress exhibit similar scaling trends, indicating that the additional runtime monitoring and adaptation mechanisms do not substantially impair scalability. All-Hash consistently achieves the highest throughput, whereas Compressed exhibits the lowest throughput due to frequent key movements and structural adjustments. Sidecar+ maintains a clear throughput advantage over the original ALEX while preserving comparable scalability to the other adaptive configurations.

*2) LIPP on Book:* Figure 19 shows that throughput increases steadily with the number of threads across all three workloads. Sidecar+, Static-Hash, and No-Compress exhibit similar scaling trends and consistently outperform the original LIPP. Concurrency also improves over the original index by decoupling rebuilding decisions, but its throughput remains lower than that of the hash-conversion configurations, particularly under INSERT and INSERT_READ. This indicates that decoupling rebuilding decisions reduces regular-operation overhead, while selective hash conversion further mitigates recursive conflict-resolution costs in degraded subtrees.

### *E. Attack-Key Injection and Resulting Distributions*

We construct poisoned workloads by injecting attack keys into indexes initialized with the preprocessed original datasets. Specifically, the original keys are first loaded into the index, after which attack keys are progressively inserted during runtime to induce distribution shifts. The injection patterns differ between the two datasets. For Book, attack keys are dispersed across the original key domain, introducing local distribution perturbations across multiple node ranges. For Covid, attack keys form dense clusters within a small number of key intervals, concentrating structural degradation on a few nodes.

Fig. 20 illustrates the resulting CDF distributions. In Book, dispersed attack keys introduce local distribution perturbations across a broad key domain, affecting multiple index regions. In contrast, Covid exhibits concentrated changes within a few key intervals, leaving most of the key domain relatively unaffected. These two patterns represent complementary distribution shifts, allowing us to evaluate Sidecar under both widespread and localized structural degradation.